九阶 · Linux 与 Python 自学资料

10

综合项目 · server-health-check

把 Linux、Python、容器与监控串成一个项目

教科书

AYMLiu-Pzw · 2026-10-22

九阶 · Linux 与 Python 自学资料

第 10 册

综合项目 · server-health-check

把 Linux、Python、容器与监控串成一个项目

资料类型	教科书
适用读者	完成 K8s 阶段 6 的学习者
发布日期	2026-10-22
作者署名	AYMLiu-Pzw

阅读导航

按章级标题定位正文；PDF 阅读器中的书签同时提供章、节两级入口。

1. 7-01 项目立项：巡检到底要检查什么
2. 7-02 v1：把四个检查写进脚本（subprocess + 阈值告警）
3. 7-03 输出报告与历史对比
4. 7-04 容器化：把巡检脚本做成镜像
5. 7-05 接到监控：把巡检指标暴露给 Prometheus
6. 7-06 README 与设计决策记录
7. 7-07 三分钟讲述稿与 10 分钟深挖
8. 7-08 阶段 7 · 学习总结练习：陌生人 clone 下来能跑

10 • 综合项目（server-health-check）

本册是阶段 7 的正文，从 7-01 起步；配套与路线见《17-关卡总表》。
本册共 8 关（7-01 ~ 7-08）。目标：交付综合项目 v1 —— 本机巡检 + 阈值告警 + 报告与历史对比 + 容器化 + 指标进监控 + README/设计决策/技术说明。
这组练习围绕项目的最终交付材料展开；做完要能在干净环境按 README 跑通。
本册是什么：综合项目 —— 把前面所有零件拼成 server-health-check：巡检、告警、报告、容器化、接监控。
动手做实验时照这两条来：实验闭环（先记录原状态 → 操作 → 按输出 / 退出码验证 → 恢复现场）和踩坑记录位（现象 → 原因 → 解决 → 一句话总结）——不必逐关都写，做完整册回头总结一次就够。

7-01 项目立项：巡检到底要检查什么

这一册以 `server-health-check` 为核心项目：训练目标是把需求、实现、证据和设计取舍串成可复核的工程说明。第一关先把"要检查什么、怎么算不正常"定下来，**不写代码**。

步骤 1：先回答一个问题

```
如果半夜收到告警"某台服务器好像不对劲"，你希望脚本替你回答哪几个问题？

1) 磁盘是不是快满了？      → df -h        （门槛：>80% 告警）
2) 内存是不是被吃光了？    → free -m      （门槛：可用 <10%）
3) CPU 负载是不是过高？    → uptime       （门槛：1 分钟负载 > 核数 × 1.5）
4) SSH 监听入口还活着吗？  → systemctl is-active ssh.socket    （门槛：不是 active 就报）
—— 这四条就是版本 1 的检查清单（和阶段 3 的 v0 相比，多了"服务检查"）
```

步骤 2：给每条定"三件事"

检查项	命令	正常 / 异常门槛	异常时说什么
磁盘	`df -h /`	使用率 ≥ 80%	"根分区用量 87%，先查是谁在涨"
内存	`free -m`	可用 ≤ 10%	"可用内存只剩 380MB"
负载	`uptime` / `nproc`	1 分钟负载 > 核数 × 1.5	"负载 7.2，超过 4 核的 1.5 倍"
SSH 监听入口	`systemctl is-active ssh.socket`	不是 `active`	"ssh.socket 状态：inactive"

解读：**每一个检查都要有"门槛"和"异常时的人话"** —— 这就是"运维脚本"和"练习脚本"的区别：前者是给人看的、能直接进告警的。

步骤 3：写一页《检查清单 + 判定标准》（产出）

```
# server-health-check 检查清单 v1
| 检查项 | 命令 | 门槛 | 异常时输出 |
|-------|------|------|-----------|
| 磁盘   | df -h / | ≥80% | [告警] 根分区 87% |
| 内存   | free -m | 可用 ≤10% | [告警] 可用内存 380MB |
| 负载   | uptime  | >核数×1.5 | [告警] 负载 7.2（4 核）|
| SSH入口 | systemctl is-active ssh.socket | ≠active | [告警] ssh.socket inactive |
退出码约定：0=全部正常，1=脚本自身出错，2=存在告警
```

完成判定

· 有一页检查清单（四项 + 门槛 + 异常话术） · 能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇） · 能说出为什么每项都要有门槛 · 能说出退出码 0/1/2 各自代表什么 · 能说出这一版和 3-17 的 v0 差在哪（多了服务检查、门槛更明确）
回滚方式：纯文档，无风险。

7-02 v1：把四个检查写进脚本（subprocess + 阈值告警）

先停在这里做最小版：合上书写出 `run(cmd)` 和 `check_disk(threshold)` 的骨架，只要求跑出“根分区使用率 xx%”和退出码 0/2 两种路径。先预测：命令不存在时由谁接住，磁盘 82% 且阈值 80 时返回什么。写出第一版后再看下方参考骨架，对照你漏掉的异常传播与退出码。

步骤 1：核心骨架（可直接复制，先跑通再改）

```
#!/usr/bin/env python3
"""server-health-check v1：四项本机巡检，超阈值告警"""
import subprocess
import argparse
import sys


def run(cmd):
    """跑一条命令，返回输出；命令不存在/失败都抛异常给上层"""
    result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
    if result.returncode != 0:
        raise RuntimeError(f"{' '.join(cmd)} 失败：{result.stderr.strip()}")
    return result.stdout


def check_disk(threshold=80):
    out = run(["df", "-h", "/"])                        # 第二行第二列是使用率
    usage = int(out.splitlines()[1].split()[4].rstrip("%"))
    return usage >= threshold, f"根分区使用率 {usage}%"
```

同一脚本续：检查函数

```
def check_memory(min_free_pct=10):
    out = run(["free", "-m"])                           # Mem 那行：total / available
    parts = out.splitlines()[1].split()
    total, available = int(parts[1]), int(parts[6])
    pct = available / total * 100
    return pct <= min_free_pct, f"可用内存 {available}MB（{pct:.0f}%）"


def check_load(factor=1.5):
    cores = int(run(["nproc"]).strip())
    load1 = float(run(["uptime"]).split("load average:")[1].split(",")[0])
    return load1 > cores * factor, f"1 分钟负载 {load1}（{cores} 核，门槛 {cores * factor}）"


def check_service(name="ssh.socket"):
    # 注意：is-active 用"返回码"表达状态（不在跑=3、不存在=4），
    # 所以这里不能用会抛异常的 run()，要自己接住返回码
    result = subprocess.run(["systemctl", "is-active", name],
                            capture_output=True, text=True, timeout=10)
    out = result.stdout.strip() or "（无输出）"
    return out != "active", f"服务 {name} 状态：{out}"


CHECKS = [
    ("磁盘", check_disk),
    ("内存", check_memory),
    ("负载", check_load),
    ("SSH入口", check_service),
]
```

同一脚本续：主流程

```
def main():
    parser = argparse.ArgumentParser(description="本机健康巡检")
    parser.add_argument("--disk-threshold", type=int, default=80,
                        help="根分区使用率告警阈值，默认 80")
    parser.add_argument("--service", default="ssh.socket",
                        help="要检查的 systemd 单元，默认 ssh.socket")
    parser.add_argument("--skip-service", action="store_true",
                        help="跳过 systemd 服务检查（容器运行时常用）")
    parser.add_argument("--serve", action="store_true",
                        help="以 /metrics 方式常驻（7-05 使用）")
    args = parser.parse_args()
    checks = [
        ("磁盘", lambda: check_disk(args.disk_threshold)),
        ("内存", check_memory),
        ("负载", check_load),
    ]
    if not args.skip_service:
        checks.append(("SSH入口", lambda: check_service(args.service)))
    alerts = []
    for name, fn in checks:
        try:
            bad, message = fn()
        except Exception as exc:
            print(f"[错误] {name} 检查失败：{exc}")
            return 1
        print(f"[{'告警' if bad else '正常'}] {name}：{message}")
        if bad:
            alerts.append(f"{name}：{message}")
    if alerts:
        print(f"\n共 {len(alerts)} 项异常")
        return 2
    print("\n全部正常")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

解读（四处值得说的设计）：① **每个检查是一个函数** —— 以后加"GPU 检查"只是再加一个函数；② **命令失败抛异常、上层统一处理**（不会静默算错）；③ **每项都打印"人话"**，不只是数字；④ **退出码 0/1/2**，可以被 crontab 或别的脚本判断。

步骤 2：跑两次 —— 正常态和告警态

```
$ chmod +x server_check.py && ./server_check.py
[正常] 磁盘：根分区使用率 42%
[正常] 内存：可用内存 3120MB（78%）
[正常] 负载：1 分钟负载 0.31（4 核，门槛 6.0）
[正常] SSH入口：服务 ssh.socket 状态：active

全部正常
（Ubuntu 26.04 常由 ssh.socket 按需拉起，因此默认检查监听入口 ssh.socket，而不是常年 inactive 的 ssh.service。若机器未安装 openssh-server，可显式传入 --service cron 或你确认应长期 active 的专用 test unit；不要把“单元不存在”误当成正常。）
$ echo $?
0

# 只在本次命令临时覆盖门槛，不改文件默认值
$ ./server_check.py --disk-threshold 1
[告警] 磁盘：根分区使用率 42%
...
共 1 项异常
$ echo $?
2
```

步骤 3：故意造一个故障场景（这步最值钱）

```
# 先不带参数运行，确认已经回到默认 threshold=80
$ ./server_check.py
[正常] 磁盘：根分区使用率 42%
...

# 在容量足够小的练习盘上，造一个大文件把使用率推过 80%
$ fallocate -l 3G ~/lab/bigfile && ./server_check.py
[告警] 磁盘：根分区使用率 81%          ← 示例；普通大磁盘上的 3G 不一定能到 80%
$ rm ~/lab/bigfile && ./server_check.py   # 清理后必须回到默认门槛下的正常态
$ echo $?
0

# 停/起同一个单元，并保存原状态
$ unit=ssh.socket
$ enabled_before=$(systemctl is-enabled "$unit")
$ active_before=$(systemctl is-active "$unit")
$ printf "操作前：enabled=%s active=%s\n" "$enabled_before" "$active_before"
$ sudo systemctl stop "$unit"
$ ./server_check.py | grep SSH入口
[告警] SSH入口：服务 ssh.socket 状态：inactive
$ sudo systemctl start "$unit"
$ printf "恢复后：enabled=%s active=%s\n" "$(systemctl is-enabled "$unit")" "$(systemctl is-active "$unit")"
```

回滚方式：实验文件用 `rm ~/lab/bigfile` 清理；服务停过一次后，必须对**同一个变量 `$unit`** 执行 start，并核对 is-active / is-enabled。**本实验要使用本地控制台，不要通过这台机器的 SSH 会话执行**，否则停掉 socket 可能连恢复命令都送不进去。若 3G 在你的磁盘上无法把使用率推过 80%，不要永久改默认阈值；可把当前使用率整数加 1 作为临时门槛做比较逻辑演练（例如当前 42% 就用 `--disk-threshold 43`），并如实记录“未造出真实 80% 磁盘状态”。

完成判定

· 脚本四项检查都能跑出结果 · 造过一次真实告警（大文件顶磁盘）并恢复 · 退出码 0/1/2 都验证过 · 完成 7-02 迁移检查：能预测并验证 `--disk-threshold 1`、`--skip-service` 的输出与退出码 · 能说清为什么每项检查用函数封装 · 能说清 `subprocess.run` 的 `capture_output` / `text` / `timeout` 各干什么

7-02 迁移检查：不改源码，分别预测并运行 `--disk-threshold 1` 与 `--skip-service`：检查项列表怎样变、退出码分别是多少、输出哪里能证明原因。再把一个检查命令临时改成不存在的名字，从“现象 → 线索 → 下一步”写出判定；完成后恢复命令。只给判断依据，不抄完整代码。

7-03 输出报告与历史对比

为什么要"留档"：只看一眼数字，你判断不了"这台机器是今天不对，还是一直都不对"。**把每次巡检写成文件，才有"趋势"**——这也是运维和临时工的区别。

步骤 1：把结果写成文件（带时间戳）

```
# 1) 在 server_check.py 顶部补这些导入；其余检查函数保持 7-02 原样。
import datetime
import os
import pathlib


# 2) 在 main() 之前加入报告函数。
REPORT_DIR = pathlib.Path(
    os.environ.get("REPORT_DIR", pathlib.Path.home() / "lab" / "health-reports")
)


def write_report(lines, alerts):
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    now = datetime.datetime.now()
    stamp = f"{now:%Y%m%d-%H%M%S-%f}"
    body = "\n".join(
        [f"巡检时间：{now:%Y-%m-%d %H:%M:%S}", f"告警数：{len(alerts)}", *lines]
    )
    for attempt in range(1, 101):
        path = REPORT_DIR / f"{stamp}-{attempt:02d}.txt"
        try:
            # "x" = 仅当文件不存在时创建，检查与创建由同一系统调用完成。
            handle = path.open("x", encoding="utf-8")
            break
        except FileExistsError:
            continue
    else:
        raise RuntimeError("无法分配不重复的报告文件名")
    with handle:
        handle.write(body + "\n")
    print(f"报告已写入：{path}")
    return path
```

解读：文件名含微秒，并用 open("x") 独占创建；同一秒甚至同一微秒内并发运行也不会互相覆盖。目录用 mkdir(parents=True, exist_ok=True) 保证第一次跑也不会因为目录不存在而失败。

步骤 2：和上一次比（这一步才是"运维视角"）

```
# 2) 继续把这些报告/对比函数和完整 main() 放到检查函数之后。
def last_reports(n=3):
    return sorted(REPORT_DIR.glob("*.txt"))[-n:]


def disk_line_from_text(text):
    return next((line for line in text.splitlines() if "根分区使用率" in line), "")


def diff_with_last(files, current_line):
    """current_line 是本轮磁盘结果；files 包含本轮和上一轮报告。"""
    if len(files) < 2:
        return "（还没有上一份报告，下一次运行再比）"
    previous = disk_line_from_text(files[-2].read_text(encoding="utf-8"))
    return f"上次：{previous or '（上一份报告没有磁盘行）'}\n本次：{current_line}"
```

同一文件续：完整 main()

```
# 用下面这版 main() 替换 7-02 的同名函数；它就是完整接线。
def main():
    parser = argparse.ArgumentParser(description="本机健康巡检")
    parser.add_argument("--disk-threshold", type=int, default=80,
                        help="根分区使用率告警阈值，默认 80")
    parser.add_argument("--service", default="ssh.socket",
                        help="要检查的 systemd 单元，默认 ssh.socket")
    parser.add_argument("--skip-service", action="store_true",
                        help="跳过 systemd 服务检查（容器运行时常用）")
    parser.add_argument("--serve", action="store_true",
                        help="以 /metrics 方式常驻（7-05 使用）")
    args = parser.parse_args()
    checks = [
        ("磁盘", lambda: check_disk(args.disk_threshold)),
        ("内存", check_memory),
        ("负载", check_load),
    ]
    if not args.skip_service:
        checks.append(("SSH入口", lambda: check_service(args.service)))

    alerts = []
    lines = []
    for name, fn in checks:
        try:
            bad, message = fn()
        except Exception as exc:
            message = f"{name} 检查失败：{exc}"
            print(f"[错误] {message}")
            lines.append(f"[错误] {message}")
            write_report(lines, alerts)
            return 1
        entry = f"[{'告警' if bad else '正常'}] {name}：{message}"
        print(entry)
        lines.append(entry)
        if bad:
            alerts.append(f"{name}：{message}")

    write_report(lines, alerts)
    current_disk = next((line for line in lines if "根分区使用率" in line), "")
    print(diff_with_last(last_reports(3), current_disk))
    if alerts:
        print(f"\n共 {len(alerts)} 项异常")
        return 2
    print("\n全部正常")
    return 0
```

解读：这版 main() 每轮把结果同时打印并收集进 `lines`，所有检查结束后调用 `write_report(lines, alerts)`；紧接着读取本轮文件与上一份文件，传入本轮磁盘行做对比。首次运行生成 1 份报告并提示“下一次再比”，第二次运行生成第 2 份并显示相邻两次结果。

解读：要点不在算法，而在**"先有历史、才有趋势"这个习惯**。生产上这叫"基线"（baseline）：没有基线，"这个数字正不正常"就只能靠猜。

步骤 3：接到定时任务上（让它自己跑）

```
$ crontab -e
0 * * * * /usr/bin/python3 /home/你的真实用户名/lab/server_check.py >> /home/你的真实用户名/lab/health.log 2>&1

# 一小时后确认：
$ tail -3 ~/lab/health.log
$ ls -1 ~/lab/health-reports | tail -3
20261009-231001-123456-01.txt
20261010-001001-223344-01.txt
20261010-011001-334455-01.txt    ← 每小时一份，文件名含微秒与独占序号
```

行里两处 /home/你的真实用户名 记得换成你的真实路径；cron 不展开 ~，写绝对路径最稳。

回滚方式：`crontab -e` 删掉那一行；报告目录 `rm -rf ~/lab/health-reports`（先确认里面没有你要留的）。

完成判定

· 每次运行都生成一份带时间戳的报告文件 · 至少跑两次，能打印出"上次 vs 本次"的对比 · 挂上 crontab，等它自己跑出至少两份 · 能说出"基线"是什么、为什么没有基线就没法判断异常

7-04 容器化：把巡检脚本做成镜像

拉不动基础镜像？（报 `i/o timeout`）按 **07 册的镜像加速器说明**配一次即可。

步骤 1：Dockerfile（照 4-05 的模板改）

```
FROM python:3.14-slim
RUN apt-get update && apt-get install -y procps tzdata && rm -rf /var/lib/apt/lists/*
ENV TZ=Asia/Shanghai
WORKDIR /opt/check
COPY server_check.py ./
RUN chmod +x server_check.py
ENTRYPOINT ["python3", "server_check.py"]
```

（**为什么装 procps**：slim 镜像里没有 `free` / `uptime`，缺什么自己装。**为什么装 tzdata**：只设置 `TZ=Asia/Shanghai` 而没有时区数据库时，Python 可能回退到 UTC；镜像和代码必须一起定义时区口径。）

解读：用 `python:3.14-slim` 当底座（比 ubuntu 小、又自带 Python）；**不用再 apt 装 Python** —— 这就是选底座的价值。

为什么用 ENTRYPOINT 而不是 CMD：镜像名后面跟的参数会**追加**到 `ENTRYPOINT`；写成 CMD 会把它**整条替换**（后面 `--skip-service` 那一步能生效就靠这个）。

构建卡在 apt-get？（国内网络常见）：容器的 apt 走 Debian 官方源，慢网下可能十几分钟拉不动。把步骤 1 的 RUN 换成下面这行（换源段会打印实际改了哪个源文件）再构建：`RUN for f in /etc/apt/sources.list.d/debian.sources /etc/apt/sources.list; do [ -f "$f" ] && echo 换源: $f && sed -i 's|deb.debian.org|mirrors.tuna.tsinghua.edu.cn|g' "$f"; done; apt-get update && apt-get install -y procps tzdata && rm -rf /var/lib/apt/lists/*`。换的只是下载通道，procps 和 tzdata 仍要安装；**注意：07 册配的加速器只管拉 Docker 镜像，管不到容器里的 apt**，两件事分开配。

步骤 2：构建、运行、看结果

```
$ docker build -t health-check:1.0 .
$ docker run --rm health-check:1.0
[正常] 磁盘：根分区使用率 1%          ← 容器 / 的 overlay 视图；数值随宿主和存储驱动变化
[正常] 内存：可用内存 8192MB（85%）     ← free 通常读宿主 /proc/meminfo，不是容器 cgroup 配额
[正常] 负载：1 分钟负载 0.10（2 核，门槛 3.0） ← 宿主内核负载视图
[错误] 服务 检查失败：[Errno 2] No such file or directory: 'systemctl'   ← 容器里没有 systemd

$ echo $?
1
```

解读：**这是一个真实且重要的坑**：容器里通常没有 systemd，所以"检查服务"这条在容器内做不了。**更要紧的是**：这条若抛错，`main()` 会返回脚本错误，后面的报告也可能写不出来。资源口径也要分清：`/` 是容器看到的 overlay 文件系统，`free` 通常读取宿主 `/proc/meminfo`，负载来自宿主内核；它们并不都表示“容器自己的资源”。若要监控容器 cgroup，应读取 `memory.max` / `memory.current` 等接口。**三种正经处理办法**：① 把服务检查做成可选（7-02 骨架已提供 `--skip-service`）；② 服务检查改用"端口探活"；③ 这个检查项留给"在宿主机上跑"的场景，容器版只做前三项。

三种都行，**关键是你要能说清为什么**——这就是"设计决策"（下一关有模板）。

本册按办法①做：7-02 的完整骨架已经用 `argparse` 定义了 `--skip-service`，并据此不加入服务检查，不需要再手工改 `sys.argv`。若你前面自己写了简化版，请把这项做成显式命令行开关，并在容器运行命令中传入。

步骤 3：把宿主机目录挂进去（报告要落地）

跑之前先建好目录：`mkdir -p ~/lab/health-reports`——目录不存在时 Docker 会自动帮你建一个，但**属主是 root**，之后你自己写、删都麻烦。

```
$ docker build -t health-check:1.0 .        # 改完脚本要重新构建
$ docker run --rm -v ~/lab/health-reports:/reports -e REPORT_DIR=/reports health-check:1.0 --skip-service
$ ls ~/lab/health-reports | tail -2       # 报告确实落在宿主机上
```

解读：**容器是"用完就删"的，报告必须写到挂载出来的目录里**（4-03 学过"无状态的放容器里、有状态的挂出来"，这里就是它的真实应用）。

报告文件名的时间比墙上的钟慢 8 小时？ `python:3.14-slim` 默认不保证含 `tzdata`，仅加 `-e TZ=Asia/Shanghai` 也不保证生效。本关 Dockerfile 已安装 `tzdata` 并设置 `ENV TZ=Asia/Shanghai`；改完 Dockerfile 要重新构建。运行时仍可显式传 `-e TZ=Asia/Shanghai`：`docker run --rm -e TZ=Asia/Shanghai -v ~/lab/health-reports:/reports -e REPORT_DIR=/reports health-check:1.0 --skip-service`。若目标机器不改镜像，代码应使用显式时区库并确保时区数据存在。

回滚方式

`docker rmi health-check:1.0` 删镜像；宿主机上的 `~/lab/health-reports` 不受影响。

完成判定

· 镜像构建成功并能跑出巡检输出 · 报告通过挂载写入宿主机目录（容器删了报告还在） · 能说清"为什么容器里 systemd 检查会失败"以及你选了哪种处理办法 · 能说出用 python:3.14-slim 当底座的好处

7-05 接到监控：把巡检指标暴露给 Prometheus

这一关让项目"活起来"：之前是"跑一次看一次"，现在是**持续暴露指标，让 Prometheus 每隔 15 秒来抓一次，超阈值还能触发告警。**这就是把脚本变成"可观测的服务"。

步骤 1：给脚本加一个 /metrics 出口（最简做法）

```
# 用标准库起一个极小的 HTTP 服务，把巡检结果按 Prometheus 格式吐出来
import re
from http.server import BaseHTTPRequestHandler, HTTPServer

LAST = {"disk": 0.0, "mem": 0.0, "load": 0.0}


def _num(text, pattern=r"([\d.]+)%"):
    """从检查函数的"人话"里抠出数字（巡检函数返回的是给人看的字符串）"""
    return float(re.search(pattern, text).group(1))


def refresh():
    """跑一次检查，把数字塞进 LAST"""
    LAST["disk"] = _num(check_disk(threshold=0)[1])                          # 根分区使用率 42% → 42.0
    LAST["mem"] = _num(check_memory(min_free_pct=0)[1], r"（([\d.]+)%）")  # 可用内存 …（78%）→ 78.0
    LAST["load"] = _num(check_load(factor=0)[1], r"负载 ([\d.]+)")          # 1 分钟负载 0.31… → 0.31
```

同一文件续：指标处理类

```
class Metrics(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path != "/metrics":
            self.send_error(404); return
        refresh()
        body = (
            "# HELP health_disk_percent 根分区使用率\n"
            "# TYPE health_disk_percent gauge\n"
            f'health_disk_percent {LAST["disk"]}\n'
            f'health_mem_available_percent {LAST["mem"]}\n'
            f'health_load1 {LAST["load"]}\n'
        )
        self.send_response(200)
        self.send_header("Content-Type", "text/plain; version=0.0.4")
        self.end_headers()
        self.wfile.write(body.encode())


# 放置位置（照这个顺序，不要追加到主守卫之后）：
# ① 把 import re 和 from http.server ... 放进文件顶部，与 argparse 等导入并列；
# ② 把 LAST / _num / refresh / Metrics 放在 main() 之前；
# ③ 在 main() 里 args = parser.parse_args() 后立刻加入下面的 serve 分支。
#     if args.serve:
#         print("指标服务已启动：http://localhost:9101/metrics（Ctrl+C 退出）")
#         HTTPServer(("0.0.0.0", 9101), Metrics).serve_forever()
#         return 0
```

解读：**Prometheus 的指标格式其实很简单** —— 一行一个"指标名 值"，前面用 `# HELP` / `# TYPE` 声明。你不需要装 prometheus_client 库也能做（装了更省事：`pip install prometheus_client` 然后用 Gauge）。

更省事的写法（推荐）：`pip install prometheus_client`，用它自带的 `start_http_server(9101)` + `Gauge("health_disk_percent", "...").set(值)` —— 格式、并发、并发安全它都替你处理。

网络边界：这个示例为了让 Docker 容器访问宿主指标，监听在 `0.0.0.0:9101`。只应在个人实验机使用，并确保 9101 不被安全组/防火墙暴露公网；若机器在共享网络，改成明确私网地址并加访问控制。示例指标服务没有鉴权。

步骤 2：起服务并验证

```
$ python3 server_check.py --serve &
$ curl -s http://localhost:9101/metrics | head -6
# HELP health_disk_percent 根分区使用率
# TYPE health_disk_percent gauge
health_disk_percent 42.0
health_mem_available_percent 78.0
health_load1 0.31
```

步骤 3：让 Prometheus 抓它 + 配一条告警

```
$ cd ~/lab/monitor
# docker-compose.yml 的 prometheus 服务下加两行（Linux / WSL 原生 Docker 必须加，
# 否则 host.docker.internal 解析不了，Target 会一直 down）：
#   extra_hosts:
#     - "host.docker.internal:host-gateway"
# prometheus.yml 的 scrape_configs 里加一段：
  - job_name: "health-check"
    static_configs:
      - targets: ["host.docker.internal:9101"]

# 在原 alerts.yml 的唯一顶层 groups: 列表里追加一项。合并后的最小结构：
groups:
  - name: disk
    rules:
      - alert: DiskUsageHigh
        expr: 100 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} *
100) > 85
        for: 1m
  - name: health-check
    rules:
      - alert: HealthDiskHigh
        expr: health_disk_percent > 80
        for: 1m
        annotations:
          summary: "巡检脚本报告：磁盘使用率 {{ $value }}%"

$ docker compose exec prometheus promtool check rules /etc/prometheus/alerts.yml
$ docker compose up -d --force-recreate prometheus
```

验证：重建后到 **Status → Targets** 确认 health-check 为 UP，再到 **Alerts** 确认新规则出现。 **解读**：**到这里，你的脚本从"一段代码"变成了"系统的一部分"**：它自己暴露指标、被监控抓取、能触发告警。注意 groups：只能有一个顶层键；health-check 是同一列表下新增的项。**单文件 bind mount 换了文件本体时，普通 restart / up -d 可能仍读旧 inode，因此本关用 `--force-recreate` prometheus 重建容器。**到这里只证明规则能进入 FIRING，本实验没有配 Alertmanager/contact point，**没有验证通知送达。**

回滚方式

停掉服务 `kill %1`；把 prometheus.yml / alerts.yml 里新增内容删掉，再执行 `docker compose up -d --force-recreate prometheus`，确认 Target 与规则恢复。

完成判定

· `/metrics` 能返回至少三个自定义指标 · Prometheus 的 Targets 里有 health-check 且 UP · Grafana 上能画出你自己的指标曲线 · 有一条自己的告警规则，且调低阈值能看到 PENDING → FIRING · 能明确说出“规则 FIRING ≠ 通知已送达”，本实验未配置通知出口 · 能说出 Prometheus 的指标格式长什么样

7-06 README 与设计决策记录

步骤 1：README（照 16 册那页的五段结构写）

```
# server-health-check

一台机器的健康巡检工具：磁盘 / 内存 / 负载 / 服务四项检查，超阈值告警，
可选容器运行，指标可被 Prometheus 抓取。

## 怎么用
    git clone https://github.com/你的用户名/server-health-check.git
    cd server-health-check
    mkdir -p ~/lab/health-reports
    python3 server_check.py
    python3 server_check.py --serve        # /metrics 常驻；Ctrl+C 退出

    docker build -t health-check:1.0 .
    docker run --rm -v ~/lab/health-reports:/reports \
      -e REPORT_DIR=/reports health-check:1.0 --skip-service

## 环境
Ubuntu 26.04 / Python 3.14；容器方式需要 Docker。本地运行时若依赖第三方包，
先按项目说明创建并激活 venv。

## 效果
    # 正常运行：本次结果全部正常
    $ python3 server_check.py
    [正常] 磁盘：根分区使用率 42%
    ...

    # 告警演练：用临时门槛触发，不改代码默认值
    $ python3 server_check.py --disk-threshold 41
    [告警] 磁盘：根分区使用率 42%
    共 1 项异常

## 踩坑记录
- 容器里没有 systemd → 服务检查改成端口探活 / --skip-service
- 报告写进容器内部 → 一删容器就没了 → 改成挂载宿主目录
```

步骤 2：设计决策记录（最值得给人看的一页）

```
# 设计决策记录

1) 为什么用 subprocess 调系统命令，而不是读 /proc？
   选了：subprocess
   其它方案：直接读 /proc/meminfo 等文件
   为什么不用：df/free/uptime 的输出就是运维日常看到的格式，一致的输出方便对照；
               /proc 格式各发行版有差异，维护成本高

2) 为什么容器版把"服务检查"做成可选？
   选了：--skip-service + 端口探活两种
   其它方案：容器里装 systemd
   为什么不用：容器里跑 systemd 是反模式（一个容器一个进程），得不偿失

3) 为什么报告按时间戳分文件，而不是一个文件追加？
   选了：每次一个文件
   其它方案：单文件追加
   为什么不用：单文件久了难检索，也容易被误删；分文件天然形成历史序列

4) 为什么自己暴露 metrics，而不是让脚本直接发告警通知？
   选了：暴露指标 + Prometheus 告警规则
   其它方案：脚本内直接发邮件/钉钉
   为什么不用：脚本只负责"产出数据"，判断和通知交给监控系统，职责更清楚，也更容易改阈值
```

解读：四条决策都按“我选了什么 / 还有什么方案 / 为什么不用”记录。这样复查项目时，不需要凭记忆重建取舍。

完成判定

· README 五段齐全，陌生人照它能跑通 · 有设计决策记录，至少覆盖四个决策 · 每条决策都写清"选了什么 / 还有什么 / 为什么不用" · 仓库里没有密钥等敏感信息

回滚方式：文档改动可随时回退（git 里都有）。

7-07 三分钟讲述稿与 10 分钟深挖

步骤 1：三分钟讲述稿（骨架照这里填，别背全文）

```
【1 分钟 · 是什么】
"这是我写的服务器健康巡检工具，检查四项：磁盘、内存、负载、关键服务。
 它可以在本机直接跑，也可以容器运行；指标能被 Prometheus 抓，超阈值会告警。"

【1 分钟 · 为什么这么做】
"我不想每次上机器手动敲四五个命令，就把它们写成脚本；
 判断交给 Prometheus，脚本只负责产出数据 —— 这样改阈值不用改代码。"

【1 分钟 · 踩了什么坑】
"容器版第一次跑就失败了：容器里没有 systemd，服务检查用不了。
 我改成两种方案：跳过参数，或者用端口探活代替。"

（结尾可补一句导航："报告的历史对比在 7-03，需要时可以展开说明。"）
```

步骤 2：准备 10 分钟技术讲解（按自检问题练习）

自检问题	回答时必须引用的项目证据
为什么不用现成的监控？（Node Exporter 不就有了）	"现成的确实更全。我写它的目的是：① 练 subprocess + 异常 + 日志这条链；② 我加了业务侧的"关键服务是否活着"这种自定义判断 —— 这正是脚本比通用 exporter 灵活的地方。"
阈值为什么定 80%?	"80% 是我按经验定的保守值，真正的做法是先跑一周看基线；这也是我加了历史对比的原因。"
脚本挂了怎么办?	"分两层：脚本自身的异常有退出码 1，Prometheus 那边如果抓不到 targets 会显示 DOWN —— 两种情况都能被发现。"
要你巡检 100 台机器呢?	"本机版先解决单机；多机我会：先用 Ansible（批量在多台机器上执行命令的工具）/SSH 批量执行（这是我下一步想学的），或者每台部署一个 —— 指标层用 Prometheus 汇总就能看到全部。"
容器版为什么不用 host 网络?	"可以用，但会占用宿主端口；我选了挂载报告目录，指标用 host.docker.internal 暴露，避免和宿主服务冲突。"

步骤 3：对着人（或录音）讲一遍

```
要求：3 分钟讲述稿不看稿；10 分钟技术讲解能回答至少 3 个自检问题
录音存进 learning-journey，文件名：server-health-check-讲解-YYYYMMDD.m4a
```

完成判定

· 三分钟讲述稿能不看稿讲完（录音为证） · 能回答至少三个自检问题 · 每个答案都能指到项目里的具体文件/命令 · 讲述稿里至少说清一个"设计取舍"

回滚方式：无操作。

7-08 阶段 7 • 学习总结练习：陌生人 clone 下来能跑

业务场景

你要把这个工具交给别人用（别人、同事、或者三个月后的自己）。**一个自己不熟悉的人，能不能只看 README 就把它跑起来？**这一关就是验收这件事。

综合要求（这册的技能全用上）

技能点：**检查清单设计（7-01）+ subprocess 与阈值告警（7-02）+ 报告与历史对比（7-03）+ Dockerfile 与挂载（7-04）+ 指标暴露与 Prometheus 告警（7-05）+ README 与设计决策（7-06）+ 三分钟讲述（7-07）**

① 仓库里四件套齐全：server_check.py、Dockerfile、README.md、设计决策记录.md；
② 在**一台干净的机器**（或新建的 Ubuntu 虚拟机/WSL）上照 README 跑一遍；
③ 记录过程中"卡住的地方"，回头把 README 补清楚（这一步最容易发现问题）；
④ 跑通三件事：本机运行出报告 / 容器运行出报告 / --serve 后 Prometheus 抓得到；
⑤ 更新讲述稿：把这次"交接给别人"发现的坑加进去。

验收

· 在干净环境照 README 跑通（截图或记录） · 三个跑法（本机 / 容器 / 指标）都验证过 · README 里补上了"我卡住过的地方" · 设计决策记录至少四条 · 三分钟讲述稿能不看稿讲完 · 仓库里无密钥、无大文件

产出

仓库 `server-health-check`（README + 脚本 + Dockerfile + 设计决策记录 + 截图）。**这是你的头号项目。**

AI 运维指向

这套东西换成 GPU 机器就是 AI 集群巡检的雏形：把"磁盘/内存/负载/服务"换成"显存/GPU 利用率/推理服务是否活着"，其余骨架一模一样 —— 8-08 你已经做过 GPU 指标那一半了。

顺带一提 • 版本 2（远程巡检）先不做

是什么：用 paramiko 通过 SSH 去巡检**别的机器**（现在这版只查本机）。
为什么留着：你入行后每天面对的是几十上百台机器，多机巡检是必然要做的；但现在做它的性价比低（先把单机版吃透更值）。
什么时候做：入行后有空时，照着 06 场景 5 的 paramiko 雏形往下写即可（那一关标着 ☆）。

本册复习卡

① 检查项要有数据源、门槛和统一退出码；报告留时间戳才能做趋势；容器里要显式处理无法使用的 systemd。
② 看到 [错误] 先查脚本自身与命令返回码；看到 [告警] 先比较实测值和门槛；告警数变化再看趋势而不是单点。
③ 独立变式：把磁盘门槛改成“当前使用率 + 1%”，跳过服务检查，先预测输出和退出码，再运行并解释变化。参考代码不再展开。